

DEVOPS MASTER COURSE

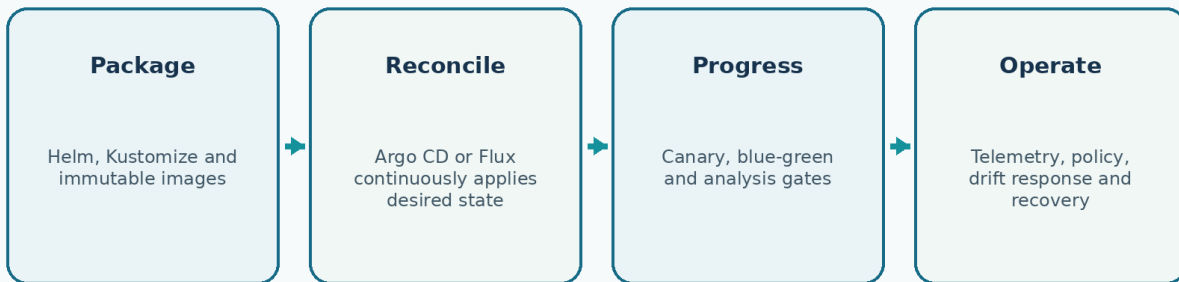
Volume 10D

Kubernetes Delivery, GitOps & Progressive Release Engineering

Kubernetes Manifests • Helm • Kustomize • Argo CD • Flux • Argo Rollouts • Policy • Observability • Enterprise Capstone

Chapters 31-40 | 2026 Edition

VOLUME 10D DELIVERY SYSTEM



Design → Build → Verify → Promote → Operate → Recover → Improve

Production objective: design and operate a secure, observable, recoverable Kubernetes delivery platform in which Git records desired state and controllers continuously reconcile verified releases.

About This Part

Part D moves CI/CD from artifact creation into Kubernetes release engineering. It explains declarative packaging, environment repositories, pull-based reconciliation, Argo CD and Flux operations, progressive delivery, policy, secrets, telemetry, multi-tenancy, and platform governance. The focus is not installing a GitOps tool; it is building an operating system for safe production change.

Graduation principle: a Kubernetes release is credible only when the exact artifact and configuration are traceable, policy-approved, observable after deployment, and recoverable without relying on one operator's memory.

Volume 10 Roadmap

Part	Coverage	Status
10A	CI/CD foundations and pipeline engineering	Complete
10B	Jenkins administration and enterprise engineering	Complete
10C	GitHub Actions and GitLab CI/CD enterprise engineering	Complete
10D	Kubernetes delivery, GitOps and progressive delivery	This book
10E	DevSecOps and software-supply-chain security	Complete
10F	Enterprise projects, troubleshooting and final capstone	Complete

Enterprise Lab Safety and Evidence Standard

- Use a disposable or explicitly approved Kubernetes cluster and synthetic data for every lab.
- Create namespaces, quotas, budgets, expiry labels, cleanup owners, and rollback criteria before deploying workloads.
- Use least-privilege service accounts, short-lived workload identity, encrypted secret delivery, and restricted cluster administration.
- Pin container images by digest for controlled promotion; never expose kubeconfig files, bearer tokens, signing material, or production identifiers in evidence.
- Define SLOs, abort thresholds, traffic-restoration steps, and evidence retention before canary, blue-green, failure-injection, or drift exercises.

How to Use This Book

1. Read the architecture and failure behavior before installing a controller.
2. Build the packaging and reconciliation path in a sandbox, then deliberately test stale state, invalid manifests, missing secrets, unhealthy workloads, and unavailable dependencies.
3. Capture Git revision, image digest, rendered manifest, controller decision, policy result, rollout evidence, telemetry, and cleanup.
4. Repeat critical recovery procedures until another engineer can execute them from the runbook.
5. Convert completed labs into a portfolio repository with sanitized diagrams, ADRs, dashboards, and incident reports.

Part Learning Outcomes

- Design a Kubernetes delivery operating model with explicit build, promotion, reconciliation, runtime, and recovery ownership.
- Package workloads with Helm and Kustomize while preserving rendered-state visibility and immutable artifact identity.
- Operate Argo CD and Flux with multi-tenancy, least privilege, observability, upgrade, backup, and clean bootstrap.
- Choose rolling, blue-green, canary, and feature-delivery strategies from workload and business risk.
- Secure GitOps repositories, controllers, secrets, policy, evidence, and emergency change as one trust chain.
- Deliver and defend a portfolio-grade Kubernetes delivery platform with measured SLOs and recovery evidence.

Table of Contents

Chapter 31	Kubernetes Delivery Operating Model and GitOps Foundations
Chapter 32	Kubernetes Manifests, Kustomize and Helm Packaging
Chapter 33	Environment Repositories, Promotion, Drift and Reconciliation
Chapter 34	Argo CD Foundations and Application Lifecycle
Chapter 35	Argo CD at Scale: ApplicationSet, Multi-Tenancy, HA and Operations
Chapter 36	Flux GitOps Toolkit, Bootstrap, Multi-Tenancy and Image Automation
Chapter 37	Deployment Strategies: Rolling, Recreate, Blue-Green, Canary and Feature Delivery
Chapter 38	Argo Rollouts, Analysis, Experiments and Traffic Management
Chapter 39	GitOps Security, Secrets, Policy, Observability and Release Governance
Chapter 40	Enterprise Kubernetes Delivery Platform Capstone
Part D Command and Investigation Reference	
Production Readiness Graduation Checklist	
Ten Additional Kubernetes Delivery Portfolio Projects	
90-Day Kubernetes Delivery Practice Plan	
Glossary	
Official Documentation Index	

Chapter 31: Kubernetes Delivery Operating Model and GitOps Foundations

Turning deployment automation into continuously reconciled, evidence-backed operations.

Why this chapter matters: Traditional push pipelines often stop after kubectl or Helm returns success. GitOps adds a persistent control loop, but only an explicit ownership and recovery model makes that loop safe.

Learning Objectives

- Explain desired, observed, and reconciled state.
- Separate build, promotion, deployment, and runtime ownership.
- Design repository, identity, evidence, and controller trust boundaries.
- Choose pull-based reconciliation where it improves control and recovery.
- Define platform SLOs and failure behavior.

Core Concepts

GitOps principles

A GitOps system uses declarative desired state, versioned and immutable history, automated pull-based reconciliation, and continuous status feedback. Git is evidence and coordination, not automatically the only source of every secret or runtime fact.

Control loops

A reconciler repeatedly compares desired and observed state. Convergence, retry, backoff, pruning, health, and deletion semantics must be understood because a wrong declaration can be applied repeatedly and at scale.

Ownership boundaries

CI builds and verifies artifacts; promotion changes approved references; a GitOps controller reconciles runtime configuration; workload operators own service health. One team may perform several roles, but evidence and authority remain distinct.

Failure domains

Repositories, credentials, controllers, clusters, namespaces, and policy services create different blast radii. Separate them according to environment criticality, tenancy, compliance, and recovery objectives.

Architecture and Operating Decisions

- Treat a Git merge as authorization only when branch policy, identity, review, and policy evidence are trustworthy.
- Do not let the deployment controller build mutable artifacts.
- Define whether manual cluster changes are forbidden, automatically reverted, or admitted only through a recorded emergency path.
- Publish controller availability, reconciliation latency, drift-detection, and recovery objectives.
- Retain source revision, artifact digest, rendered configuration, policy decision, sync result, and health evidence.

Reference Configuration or Command Pattern

```
# conceptual promotion record
release:
  application: payments-api
  image_digest: sha256:<digest>
  config_revision: <git-sha>
  environment: production
  approved_by: <change-identity>
  verification: [policy, smoke, slo]
```

```
rollback_to: <previous-release>
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Git says deployed but service is unavailable	Sync success was treated as operational validation	Add health, smoke, SLO and dependency checks	Controller status plus user-path test
Manual fixes disappear	Controller correctly reconciles an undocumented hotfix	Use an emergency commit or pause with approval, then reconcile	Incident record and final Git revision
Controller changes too much	Scope or pruning policy is broader than intended	Stop reconciliation, inspect diff, restore known revision and narrow scope	Affected resources and recovery timeline
Teams bypass GitOps	Golden path is slow or unsupported	Measure friction, add supported exceptions and improve self-service	Adoption and bypass audit

Hands-on Lab 31: Create a GitOps Operating Contract

Prerequisites

- A disposable Kubernetes cluster with kubectl access.
- A Git repository and a sample container image in a registry.
- Permission to create a dedicated namespace and inspect controller logs.

Implementation

1. Map source, build, registry, promotion repository, controller, policy engine, cluster, telemetry, and operators.
2. Define who can propose, approve, reconcile, pause, override, and recover production change.
3. Write a desired-state contract with artifact identity, environment, policy, health, and rollback fields.
4. Create a repository change and observe the controller reconciliation sequence.
5. Introduce controlled drift and document detection and correction.
6. Simulate an invalid desired state and execute the recovery runbook.
7. Publish SLOs, escalation, evidence retention, and exception handling.

Validation Evidence

- The declared revision and image digest are visible in Git and runtime evidence.
- Healthy-state and failure-state results are captured.
- Rollback or recovery meets the stated objective.

Cleanup

- Delete temporary namespaces, releases, repositories, credentials, and test records.
- Retain only sanitized diagrams, logs, decisions, and validation evidence.

Interview Questions and Model Answers

1. What is the difference between CI/CD and GitOps?

CI/CD is the broader delivery system; GitOps is an operating model for declarative state and automated reconciliation, usually covering deployment and configuration rather than artifact compilation.

2. Is Git always the source of truth?

Git is the source of declared intent. Runtime health, secret values, inventory, and external system state may require other authoritative systems.

3. Why can reconciliation be dangerous?

A controller applies intent repeatedly and quickly; excessive scope, bad pruning, or an incorrect declaration can amplify error.

4. What proves a GitOps release succeeded?

Traceable desired state plus policy, reconciliation, workload health, user-path validation, and recoverability evidence.

Chapter Summary

- GitOps is an operating model, not a product installation.
- Build, promotion, reconciliation, and runtime ownership must be explicit.
- Continuous control loops require bounded scope and tested recovery.
- Operational success is measured beyond sync status.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 32: Kubernetes Manifests, Kustomize and Helm Packaging

Building reusable deployment packages without hiding the rendered state.

Why this chapter matters: Poor packaging creates copied manifests, uncontrolled templating, inconsistent defaults, and reviews that cannot predict the final Kubernetes objects.

Learning Objectives

- Design stable Kubernetes resource contracts.
- Use Kustomize bases and overlays appropriately.
- Build Helm charts with schemas, tests, and safe defaults.
- Validate rendered manifests before deployment.
- Version application and configuration independently.

Core Concepts

Raw manifests

Plain YAML is transparent and useful for small systems, but duplication grows quickly. Resource names, selectors, labels, probes, limits, security contexts, disruption controls, and ownership metadata should follow platform conventions.

Kustomize

Bases express reusable intent; overlays apply environment-specific patches. Keep overlays small and avoid a maze of replacements that makes final state difficult to review.

Helm

Charts package templates, values, metadata, dependencies, and lifecycle hooks. Values schemas, linting, template tests, immutable chart versions, and explicit upgrade behavior reduce surprise.

Rendered-state validation

Review the final objects, not only templates. Validate syntax, API compatibility, policy, image references, selector consistency, immutable-field changes, and diffs against the target cluster.

Architecture and Operating Decisions

- Choose one packaging owner and versioning policy.
- Keep environment variability in data rather than branching template logic wherever possible.
- Prohibit floating image tags in controlled environments.
- Avoid hooks for ordinary lifecycle when native controllers can represent the dependency.
- Store rendered or reproducible evidence for high-risk releases.

Reference Configuration or Command Pattern

```
# Render and validate before merge
kubectl kustomize overlays/prod > rendered.yaml
kubectl apply --server-side --dry-run=server -f rendered.yaml
kubectl diff -f rendered.yaml

helm lint charts/payments
helm template payments charts/payments -f values-prod.yaml > helm-rendered.yaml
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.

- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Deployment selects no pods	Labels and selectors diverged during templating	Compare rendered selector/labels and add schema tests	Rendered objects and endpoint readiness
Upgrade fails on immutable field	Template changed a field requiring replacement	Plan migration, new resource name or controlled recreation	Diff and rollback result
Environment overlay is unreadable	Too many patches and replacements	Refactor base boundaries and reduce variance	Smaller rendered diff
Helm value silently ignored	Wrong key or missing schema validation	Add values.schema.json and fail unknown inputs	Lint and negative test

Hands-on Lab 32: Build and Test a Reusable Kubernetes Package

Prerequisites

- A disposable Kubernetes cluster with kubectl access.
- A Git repository and a sample container image in a registry.
- Permission to create a dedicated namespace and inspect controller logs.

Implementation

1. Create a base workload with ServiceAccount, Deployment, Service, ConfigMap, PDB, probes, limits, and security context.
2. Add development and production Kustomize overlays with only justified variance.
3. Build an equivalent Helm chart with values schema and NOTES documentation.
4. Render both variants and compare the final Kubernetes objects.
5. Run client and server dry-run, policy validation, and a selector consistency test.
6. Deploy one package, upgrade it, then roll back.
7. Record packaging trade-offs in an ADR.

Validation Evidence

- The declared revision and image digest are visible in Git and runtime evidence.
- Healthy-state and failure-state results are captured.
- Rollback or recovery meets the stated objective.

Cleanup

- Delete temporary namespaces, releases, repositories, credentials, and test records.
- Retain only sanitized diagrams, logs, decisions, and validation evidence.

Interview Questions and Model Answers

1. When is Kustomize preferable to Helm?

When the team needs transparent layering and patching of native manifests without a template language or package lifecycle.

2. Why validate rendered manifests?

Templates and values are indirect; only rendered objects show what the API server will evaluate.

3. Why are floating image tags risky?

The same declaration can resolve to different bytes, breaking auditability and rollback.

4. What is a dangerous Helm hook?

A hook that owns critical lifecycle outside normal controllers without idempotency, observability, or recovery.

Chapter Summary

- Package contracts should be predictable and reviewable.
- Kustomize manages layered variance; Helm manages parameterized packages and lifecycle.
- Schemas, rendering, dry-runs, policy and diffs catch different failure classes.
- Immutable artifacts and explicit configuration versions enable promotion.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 33: Environment Repositories, Promotion, Drift and Reconciliation

Designing repository topology and release flow across environments and clusters.

Why this chapter matters: A GitOps controller cannot compensate for ambiguous repository ownership, uncontrolled promotion, stale branches, or environment copies that diverge without explanation.

Learning Objectives

- Compare monorepo, multirepo, branch, directory, and repository-per-environment models.
- Design promotion as an auditable change of immutable references.
- Detect and classify drift.
- Control pruning, deletion, dependency order, and reconciliation cadence.
- Prevent environment divergence.

Core Concepts

Repository topology

Topology should follow ownership, access boundaries, change coupling, and blast radius. A single repository simplifies visibility; separated environment or tenant repositories can strengthen authorization and recovery boundaries.

Promotion

Promotion should change an immutable artifact or bundle reference and preserve evidence from build through production. Rebuilding for each environment invalidates the assurance created by earlier tests.

Drift classes

Intentional emergency change, unauthorized mutation, controller lag, external operator ownership, defaulting, and nondeterministic rendering require different responses. Not every difference should be blindly overwritten.

Ordering and pruning

Namespaces, CRDs, operators, secrets, migrations, workloads, and policies may have dependencies. Pruning can delete critical resources; ordering must be explicit and failure-safe.

Architecture and Operating Decisions

- Keep environment-specific approvals at the promotion boundary.
- Automate pull or merge requests rather than direct production writes.
- Classify resources owned by GitOps versus other controllers.
- Use deletion safeguards for namespaces, persistent data, and shared resources.
- Measure reconciliation age and stale desired-state sources.

Reference Configuration or Command Pattern

```
environments/
  dev/apps/payments.yaml
  staging/apps/payments.yaml
  prod/apps/payments.yaml

# promotion changes only the immutable digest
image:
  repository: registry.example/payments
  digest: sha256:<verified-digest>
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Production differs from staging	Environment copies accumulated unreviewed variance	Generate or constrain overlays and compare promotion diffs	Cross-environment diff and ADR
Deleted Git file removes data	Pruning lacked protection for stateful resources	Disable broad prune, add finalizers/approval and restore	Deletion audit and recovery
Drift alert never clears	Another controller owns the field or defaulting changes it	Define field ownership and ignore only proven noise	Managed-field analysis
Promotion uses new build	Pipeline rebuilds instead of promoting digest	Promote registry identity and attestations unchanged	Matching digest across environments

Hands-on Lab 33: Design a Multi-Environment Promotion Repository

Prerequisites

- A disposable Kubernetes cluster with kubectl access.
- A Git repository and a sample container image in a registry.
- Permission to create a dedicated namespace and inspect controller logs.

Implementation

1. Create development, staging, and production environment directories or repositories.
2. Define ownership, branch protection, promotion authority, and emergency change policy.
3. Deploy one immutable digest to development and promote it without rebuilding.
4. Introduce safe, unsafe, and controller-owned drift and classify each response.
5. Test pruning of an ordinary resource and protected handling of a stateful resource.
6. Measure source age, reconciliation latency, and environment divergence.
7. Publish a promotion and rollback runbook.

Validation Evidence

- The declared revision and image digest are visible in Git and runtime evidence.
- Healthy-state and failure-state results are captured.
- Rollback or recovery meets the stated objective.

Cleanup

- Delete temporary namespaces, releases, repositories, credentials, and test records.
- Retain only sanitized diagrams, logs, decisions, and validation evidence.

Interview Questions and Model Answers

1. What should promotion change?

An approved immutable artifact reference and the minimum environment configuration required for that release.

2. Why not use one branch per environment by default?

Long-lived branches diverge, hide differences, and make merges and audit harder; use them only with explicit controls.

3. Should all drift be reverted?

No. First identify ownership and intent; emergency changes and fields managed by other controllers need controlled reconciliation.

4. What is the risk of prune?

A stale or malicious declaration can delete live resources, including data-bearing objects, if scope and safeguards are weak.

Chapter Summary

- Repository topology is an authorization and blast-radius decision.
- Promotion preserves artifact identity.
- Drift needs ownership-aware classification.
- Pruning and ordering require explicit safety design.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 34: Argo CD Foundations and Application Lifecycle

Installing, configuring, synchronizing, observing, and recovering Argo CD Applications.

Why this chapter matters: Argo CD makes desired state visible and continuously reconciled, but unsafe projects, credentials, sync options, or health assumptions can expand deployment risk.

Learning Objectives

- Explain Argo CD components and Application resources.
- Configure repositories, destinations, projects, sync, health, hooks, waves, and rollback.
- Use SSO and least privilege.
- Troubleshoot source, comparison, sync, and health failures.
- Back up and rebuild controller configuration.

Core Concepts

Argo CD architecture

API server, repository server, application controller, Redis, and optional ApplicationSet controller have different trust and scaling characteristics. Repository rendering is a high-risk boundary because it processes untrusted configuration.

Applications and projects

An Application binds source, path/chart, revision, destination, and sync policy. AppProjects restrict repositories, clusters, namespaces, resource kinds, roles, and orphaned resources.

Sync and health

Automated sync, self-heal, prune, waves, hooks, retry, and selective sync change failure behavior. Health assessment must represent operational readiness rather than object existence.

Recovery

Controller state is largely declarative, but secrets, repository credentials, cluster registrations, SSO, encryption keys, and customizations require protected backup or reproducible bootstrap.

Architecture and Operating Decisions

- Use AppProjects as deny-by-default tenant boundaries.
- Separate Argo CD administration from application ownership.
- Pin source revisions and plugins; restrict custom tooling in repo-server.
- Enable automated sync only after rollback and deletion behavior are proven.
- Observe reconciliation queue, API latency, repository errors, cache, and cluster connectivity.

Reference Configuration or Command Pattern

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: payments-prod
  namespace: argocd
spec:
  project: payments
  source:
    repoURL: https://git.example/platform/env-prod.git
    targetRevision: main
    path: apps/payments
  destination:
    server: https://kubernetes.default.svc
    namespace: payments
  syncPolicy:
    automated: {prune: true, selfHeal: true}
```

```
syncOptions: [CreateNamespace=true]
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Application is OutOfSync forever	Rendering nondeterminism, ignored fields, mutation or failed comparison	Inspect normalized diff and managed fields; remove nondeterminism	Stable repeated comparison
Sync succeeds but remains Degraded	Health rule or workload readiness failure	Inspect resource tree, events and health customization	Healthy user-path test
Repository cannot be read	Credential, CA, DNS, proxy or rate-limit issue	Test from repo-server and rotate scoped credentials	Connection and audit logs
Argo CD cannot manage cluster	Registration secret, RBAC or network path failed	Validate identity, cluster permissions and connectivity	Least-privilege access test

Hands-on Lab 34: Operate an Argo CD Application Safely

Prerequisites

- A disposable Kubernetes cluster with kubectl access.
- A Git repository and a sample container image in a registry.
- Permission to create a dedicated namespace and inspect controller logs.

Implementation

1. Install Argo CD in a sandbox with an ingress or port-forward restricted to the lab.
2. Configure SSO or a lab identity, project-scoped RBAC, repository credential, and target namespace.
3. Create an Application and perform manual sync, automated sync, self-heal, and prune tests.
4. Use sync waves or hooks for a controlled prerequisite.
5. Break repository access and workload readiness; diagnose both.
6. Roll back to a previous revision and verify the service.
7. Export declarative configuration and rehearse rebuild.

Validation Evidence

- The declared revision and image digest are visible in Git and runtime evidence.
- Healthy-state and failure-state results are captured.
- Rollback or recovery meets the stated objective.

Cleanup

- Delete temporary namespaces, releases, repositories, credentials, and test records.
- Retain only sanitized diagrams, logs, decisions, and validation evidence.

Interview Questions and Model Answers

1. What is the purpose of an AppProject?

It limits the sources, destinations, resource kinds, and roles available to a group of Applications.

2. Does Synced mean healthy?

No. Synced means observed resources match desired state; health and user experience require separate evidence.

3. Why restrict repo-server plugins?

They execute during manifest generation and can become a code-execution and credential-exposure boundary.

4. What should be backed up?

Declarative controller configuration plus protected credentials, registrations, keys, SSO and any nonreproducible state needed for bootstrap.

Chapter Summary

- Applications connect source and destination through a project policy boundary.
- Sync and health are separate dimensions.
- Repository rendering and controller credentials deserve strong isolation.
- Rebuild and rollback must be rehearsed.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 35: Argo CD at Scale: ApplicationSet, Multi-Tenancy, HA and Operations

Operating fleets of applications and clusters without creating one global blast radius.

Why this chapter matters: Scaling the number of Applications is easy; scaling ownership, queueing, isolation, upgrades, credentials, and incident response is the enterprise challenge.

Learning Objectives

- Generate Applications safely with ApplicationSet.
- Design multi-tenancy and cluster onboarding.
- Scale and harden Argo CD components.
- Define upgrade, backup, SLO, and disaster-recovery procedures.
- Operate thousands of reconciliation objects with bounded failure domains.

Core Concepts

ApplicationSet

Generators create Applications from Git, clusters, pull requests, matrices, lists, or SCM metadata. Generator inputs are deployment authority and require review, validation, and scope limits.

Tenancy models

Shared instance, instance per trust zone, namespace-scoped patterns, and cluster-local instances trade centralization for isolation. Choose boundaries from access, compliance, scale, and recovery needs.

High availability and scale

Repository rendering CPU/memory, controller sharding, reconciliation queues, cache, API limits, webhook storms, and cluster latency constrain throughput. HA does not remove shared-dependency risk.

Operations

Upgrades must preserve CRD compatibility, custom health rules, plugins, RBAC, SSO, and client behavior. Define maintenance, rollback, backups, bootstrap, and a tested clean-room rebuild.

Architecture and Operating Decisions

- Partition by trust and failure domain rather than application count alone.
- Require review of generator templates and cluster labels used for targeting.
- Apply resource quotas and concurrency limits to protect controller dependencies.
- Test upgrade and downgrade paths in a representative environment.
- Track reconciliation latency percentiles, queue depth, errors, stale apps, API throttling, and tenant impact.

Reference Configuration or Command Pattern

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata: {name: cluster-addons}
spec:
  generators:
    - clusters:
        selector:
          matchLabels: {environment: production}
  template:
    metadata: {name: '{{name}}-addons'}
    spec:
      project: platform
      source: {repoURL: https://git.example/addons.git, targetRevision: main, path:
overlays/{{metadata.labels.region}}}
      destination: {server: '{{server}}', namespace: platform-addons}
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Only some clusters update	Generator labels, cache or cluster credential is stale	Inspect generated Applications and cluster secret metadata	Expected fleet coverage
Controller queue grows	Reconciliation burst, slow repositories or API throttling	Shard, stagger, cache, tune concurrency and protect dependencies	Queue latency returns to SLO
Tenant can deploy cluster-scoped resource	Project or RBAC boundary is too broad	Restrict kinds, destinations and controller scope	Negative authorization test
Upgrade causes widespread errors	CRD/plugin/customization incompatibility	Pause rollout, restore version/config and test migration	Recovery and compatibility report

Hands-on Lab 35: Build a Multi-Tenant Argo CD Fleet

Prerequisites

- A disposable Kubernetes cluster with kubectl access.
- A Git repository and a sample container image in a registry.
- Permission to create a dedicated namespace and inspect controller logs.

Implementation

1. Register at least two sandbox clusters or simulated destinations with labels.
2. Create AppProjects for platform and application tenants with deny-by-default boundaries.
3. Generate Applications with an ApplicationSet and validate target selection.
4. Add quotas, sync windows, RBAC, SSO groups, and notification ownership.
5. Generate a reconciliation burst and observe queue and resource behavior.
6. Simulate one unavailable cluster and prove isolation from other tenants.
7. Run a controlled Argo CD upgrade and recovery rehearsal.

Validation Evidence

- The declared revision and image digest are visible in Git and runtime evidence.
- Healthy-state and failure-state results are captured.
- Rollback or recovery meets the stated objective.

Cleanup

- Delete temporary namespaces, releases, repositories, credentials, and test records.
- Retain only sanitized diagrams, logs, decisions, and validation evidence.

Interview Questions and Model Answers

1. Why is ApplicationSet high impact?

A small generator or template change can create, retarget, or delete many Applications across clusters.

2. When use multiple Argo CD instances?

When trust, compliance, availability, scale, network, or recovery boundaries justify independent control planes.

3. What is an important Argo CD SLO?

Time from approved desired-state change to verified healthy reconciliation, measured by environment and tenant.

4. Does HA guarantee resilience?

No. Shared repositories, identity, DNS, storage, or API dependencies can still create correlated failure.

Chapter Summary

- Fleet generation is deployment authority.
- Tenancy and failure boundaries must be deliberate.
- Scale is constrained by rendering, queues, APIs, and dependencies.
- Upgrades and clean-room recovery require evidence.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 36: Flux GitOps Toolkit, Bootstrap, Multi-Tenancy and Image Automation

Composing source, reconciliation, notification, and automation controllers into a secure delivery system.

Why this chapter matters: Flux is modular and highly declarative; that flexibility requires clear dependency ordering, service-account scope, source trust, and recovery design.

Learning Objectives

- Explain Flux source, kustomize, helm, notification, and image controllers.
- Bootstrap a cluster reproducibly.
- Design Kustomization dependencies and multi-tenancy.
- Use image automation without bypassing promotion controls.
- Troubleshoot reconciliation and source failures.

Core Concepts

Toolkit architecture

Source controllers fetch Git, OCI, Helm, and bucket content; Kustomize and Helm controllers reconcile workloads; notification controllers emit and receive events; image controllers discover and update references.

Bootstrap

Bootstrap installs controllers and writes their declarative configuration to Git. Protect the bootstrap identity and repository because it establishes continuing deployment authority.

Dependencies and tenancy

Kustomization dependsOn, health checks, service accounts, namespaces, source restrictions, and cluster policies shape ordering and tenant blast radius.

Image automation

Image scanning and update automation can create reviewed Git changes. Production promotion should still require immutable identity, policy evidence, and environment authorization rather than automatically tracking the newest tag.

Architecture and Operating Decisions

- Prefer OCI or Git sources with verified immutable references where practical.
- Use serviceAccountName and namespace boundaries to reduce controller privilege.
- Separate platform and tenant Kustomizations and order only real dependencies.
- Require pull requests for high-risk image updates.
- Monitor source freshness, artifact fetch, reconciliation duration, suspend state, dependency readiness, and notification delivery.

Reference Configuration or Command Pattern

```
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata: {name: payments, namespace: flux-system}
spec:
  interval: 5m
  path: ./clusters/prod/payments
  prune: true
  sourceRef: {kind: GitRepository, name: platform}
  serviceAccountName: payments-reconciler
  dependsOn: [{name: platform-policies}]
  healthChecks:
    - {apiVersion: apps/v1, kind: Deployment, name: payments, namespace: payments}
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Kustomization not ready	Source artifact, dependency, decryption, validation or apply failed	Use flux get/trace and inspect controller event chain	Ready condition and applied revision
Old revision remains deployed	Source interval, webhook, cache or suspended object	Reconcile source, inspect suspend and artifact revision	Current Git revision observed
Tenant modifies shared namespace	Reconciler service account has excess privilege	Narrow RBAC, namespace and source scope	Denied negative test
Image automation promotes unsafe tag	Policy selected newest semver without release evidence	Use digest and reviewed promotion constraints	Approved commit and attestation

Hands-on Lab 36: Bootstrap and Operate a Flux Cluster

Prerequisites

- A disposable Kubernetes cluster with kubectl access.
- A Git repository and a sample container image in a registry.
- Permission to create a dedicated namespace and inspect controller logs.

Implementation

1. Bootstrap Flux to a dedicated repository using a least-privilege lab identity.
2. Create source and Kustomization objects for platform and application layers.
3. Add dependsOn, health checks, service-account impersonation, and notifications.
4. Introduce a stale source and broken dependency; trace the conditions.
5. Configure image discovery and generate a pull request rather than direct production update.
6. Suspend and resume reconciliation under a recorded change procedure.
7. Delete and rebuild the controller from Git and protected bootstrap inputs.

Validation Evidence

- The declared revision and image digest are visible in Git and runtime evidence.
- Healthy-state and failure-state results are captured.
- Rollback or recovery meets the stated objective.

Cleanup

- Delete temporary namespaces, releases, repositories, credentials, and test records.
- Retain only sanitized diagrams, logs, decisions, and validation evidence.

Interview Questions and Model Answers

1. Why is Flux called a toolkit?

It consists of composable controllers for sources, Kustomize, Helm, notification, and image automation rather than one monolithic application model.

2. What does bootstrap establish?

The controller installation, repository linkage, and ongoing declarative management of the Flux control plane.

3. Why use service-account impersonation?

It limits each Kustomization or HelmRelease to the permissions required for its tenant or workload.

4. Should image automation update production automatically?

Only when the selection, verification, authorization, and rollback model makes that risk acceptable; many organizations use reviewed promotion changes.

Chapter Summary

- Flux composes several controllers and condition chains.
- Bootstrap credentials and repositories are high-trust assets.
- Dependencies and impersonation establish safer tenancy.
- Image automation must preserve promotion governance.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 37: Deployment Strategies: Rolling, Recreate, Blue-Green, Canary and Feature Delivery

Choosing release mechanics from workload behavior, data constraints, and business risk.

Why this chapter matters: No deployment strategy is universally safest. A strategy must reflect compatibility, capacity, session behavior, data migration, observability, and rollback cost.

Learning Objectives

- Compare Kubernetes deployment strategies and traffic controls.
- Plan backward-compatible schema and API change.
- Define capacity, surge, disruption, and rollback behavior.
- Use feature flags without creating permanent operational debt.
- Select strategies from measurable risk.

Core Concepts

Rolling and recreate

RollingUpdate replaces replicas according to maxSurge and maxUnavailable; Recreate stops old pods first. Readiness, termination, connection draining, startup time, and capacity determine user impact.

Blue-green

Two complete versions exist while a Service, Gateway, ingress, or external router selects active traffic. It offers fast traffic reversal but requires duplicate capacity and careful state compatibility.

Canary

A new version receives limited traffic or users while metrics are compared. Safe canaries need representative sampling, enough duration, guardrails against low traffic, and a trustworthy baseline.

Feature delivery

Flags separate deployment from exposure, enabling cohorts and rapid disablement. Flag ownership, expiry, evaluation failure, privacy, consistency, and test combinatorics must be managed.

Architecture and Operating Decisions

- Choose strategy per workload rather than by platform default alone.
- Require backward and forward compatibility across the rollback window.
- Include dependency, queue, cache, job, and database behavior in rollback design.
- Reserve sufficient capacity for surge, duplicate environments, and failure recovery.
- Define business and technical success metrics before release.

Reference Configuration or Command Pattern

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 0
  minReadySeconds: 20
  progressDeadlineSeconds: 600
# Pair with readiness, PDB, graceful termination and SLO validation.
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.

- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Rolling update stalls	Readiness never succeeds or capacity cannot schedule surge	Inspect events, quotas, probes and dependency readiness	All replicas available within deadline
Rollback restores code but not service	Schema or external side effect is incompatible	Use expand/contract, compensation or fix-forward	Data and transaction validation
Canary looks healthy but users fail	Traffic sample excludes critical cohorts or dependencies	Add route, tenant and business-segment analysis	Representative transaction metrics
Feature flag outage breaks requests	Evaluation is synchronous with no safe default	Cache decisions, define default and isolate provider	Failure test with expected behavior

Hands-on Lab 37: Qualify Three Deployment Strategies

Prerequisites

- A disposable Kubernetes cluster with kubectl access.
- A Git repository and a sample container image in a registry.
- Permission to create a dedicated namespace and inspect controller logs.

Implementation

1. Create a sample API with readiness, graceful shutdown, metrics, and a backward-compatible response contract.
2. Run a rolling update and measure availability, surge, termination, and progress deadline.
3. Implement blue-green switching and prove rapid traffic reversal.
4. Implement a manual canary with a bounded traffic cohort and comparison dashboard.
5. Introduce an incompatible database change and redesign it using expand/contract.
6. Add a feature flag with owner, expiry, safe default, and outage behavior.
7. Write a strategy decision matrix for three workload types.

Validation Evidence

- The declared revision and image digest are visible in Git and runtime evidence.
- Healthy-state and failure-state results are captured.
- Rollback or recovery meets the stated objective.

Cleanup

- Delete temporary namespaces, releases, repositories, credentials, and test records.
- Retain only sanitized diagrams, logs, decisions, and validation evidence.

Interview Questions and Model Answers

1. What determines maxUnavailable?

Availability objective, replica count, failure reserve, capacity, and the service's ability to tolerate reduced instances.

2. Why can rollback be impossible?

Data, messages, external side effects, or clients may have moved beyond the old version's compatibility boundary.

3. What makes a canary representative?

Traffic, tenants, regions, routes, dependencies, payloads, and duration reflect meaningful production risk.

4. What is feature-flag debt?

Stale flags and branching behavior that increase complexity, testing combinations, and incident ambiguity.

Chapter Summary

- Strategy follows workload and business risk.
- Compatibility is the foundation of safe rollback.
- Capacity and observability are release controls.
- Feature flags need lifecycle governance.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 38: Argo Rollouts, Analysis, Experiments and Traffic Management

Automating progressive delivery with measurable promotion and abort decisions.

Why this chapter matters: Progressive delivery can reduce blast radius only when traffic routing, metric queries, baselines, thresholds, and fallback behavior are trustworthy.

Learning Objectives

- Model Rollout, ReplicaSet, Service, AnalysisTemplate, AnalysisRun, and Experiment behavior.
- Implement canary and blue-green strategies.
- Integrate ingress, service mesh, or Gateway traffic routing.
- Design analysis metrics and inconclusive behavior.
- Recover from controller, metric, and routing failures.

Core Concepts

Rollout controller

Argo Rollouts extends Kubernetes with strategies, steps, pauses, service switching, traffic routing, and analysis. The controller becomes part of the production release control plane.

Analysis

AnalysisRuns execute metric providers or jobs and classify success, failure, error, or inconclusive. Queries need stable dimensions, correct windows, baseline comparison, and protection from missing or delayed data.

Experiments

Experiments run candidate and baseline ReplicaSets for controlled comparison. They are useful when stable and canary must coexist under the same observation window.

Traffic management

Ingress, service mesh, ALB, NGINX, Gateway API, or custom routing integrations must agree with Kubernetes replica state. Routing propagation, session affinity, and fallback are part of correctness.

Architecture and Operating Decisions

- Make analysis templates versioned and owned like production code.
- Define failure, error, inconclusive, timeout, and provider-outage behavior.
- Use business metrics alongside infrastructure metrics.
- Protect stable Services and traffic-routing objects from unrelated ownership.
- Retain the analysis result, metric query, rollout revision, traffic weights, and abort decision.

Reference Configuration or Command Pattern

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata: {name: payments}
spec:
  replicas: 8
  strategy:
    canary:
      stableService: payments-stable
      canaryService: payments-canary
      steps:
        - setWeight: 10
        - pause: {duration: 5m}
        - analysis:
            templates: [{templateName: payments-slo}]
        - setWeight: 50
```

```
- pause: {}
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Rollout pauses forever	Manual pause, missing AnalysisRun or controller issue	Inspect rollout tree, conditions and analysis ownership	Explicit resume or abort record
Analysis passes on no data	Query returns empty/default success	Fail or mark inconclusive on insufficient samples	Negative low-traffic test
Traffic weight differs from pods	Router propagation or integration mismatch	Inspect provider objects and endpoint distribution	Observed request share
Abort leaves unexpected version active	Service selector or router state not restored	Execute abort verification and force stable route safely	User-path and routing evidence

Hands-on Lab 38: Build an Automated Canary with Abort

Prerequisites

- A disposable Kubernetes cluster with kubectl access.
- A Git repository and a sample container image in a registry.
- Permission to create a dedicated namespace and inspect controller logs.

Implementation

1. Install Argo Rollouts and its kubectl plugin in a sandbox.
2. Create stable and canary Services, a Rollout, and a versioned AnalysisTemplate.
3. Integrate a supported traffic router or simulate bounded header-based traffic.
4. Deploy a healthy candidate and observe staged promotion.
5. Deploy a candidate that violates latency or business-success thresholds and confirm automatic abort.
6. Simulate unavailable metrics and verify the chosen inconclusive behavior.
7. Capture traffic, ReplicaSet, analysis, abort, and stable-service evidence.

Validation Evidence

- The declared revision and image digest are visible in Git and runtime evidence.
- Healthy-state and failure-state results are captured.
- Rollback or recovery meets the stated objective.

Cleanup

- Delete temporary namespaces, releases, repositories, credentials, and test records.
- Retain only sanitized diagrams, logs, decisions, and validation evidence.

Interview Questions and Model Answers

1. Why not analyze only CPU?

CPU is an implementation signal; user success, errors, latency, and business outcomes are closer to release risk.

2. What should happen when metrics are unavailable?

The explicit risk policy should pause, abort, or require approval; absence of data must not silently equal success.

3. What is an Experiment?

A controlled run of one or more ReplicaSets, often stable and candidate, with optional analysis for comparison.

4. What proves an abort worked?

Traffic returns to the stable version, candidate exposure stops, service health recovers, and the decision is recorded.

Chapter Summary

- Progressive delivery depends on trustworthy metrics and routing.
- Analysis states must include missing and inconclusive evidence.
- Traffic and replica state must be verified together.
- Abort is a tested operational path, not a UI button.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 39: GitOps Security, Secrets, Policy, Observability and Release Governance

Protecting the full path from desired-state change to reconciled production behavior.

Why this chapter matters: A GitOps system centralizes high-impact change through repositories, controllers, identities, secrets, policy, and cluster APIs; compromise at any boundary can propagate trusted-looking changes.

Learning Objectives

- Threat-model a GitOps delivery chain.
- Deliver secrets without plaintext Git exposure.
- Apply policy before merge and at admission.
- Design controller, release, and evidence observability.
- Govern exceptions, emergency change, and audit.

Core Concepts

Threat model

Protect contributor identity, branch policy, repository administration, webhooks, CI promotion, controller credentials, rendering plugins, image registries, policy engines, cluster APIs, and evidence stores as one trust chain.

Secret delivery

Use external secret managers, workload identity, encrypted declarative references, or sealed/encrypted values with well-defined key custody and rotation. Git should contain references or ciphertext, not recoverable plaintext.

Policy layers

Static validation gives fast feedback; server dry-run adds API context; admission enforces cluster truth; runtime detection identifies behavior policy cannot prevent. Version policies and test positive, negative, outage, and exception paths.

Observability and governance

Measure source freshness, reconciliation latency, failure rate, drift, unauthorized changes, policy decisions, controller privilege use, secret errors, rollout health, and evidence completeness. Exceptions require scope, owner, compensating controls, expiry, and review.

Architecture and Operating Decisions

- Use protected workload identity rather than long-lived cluster credentials.
- Sign or attest promoted artifacts and verify immutable identity before deployment.
- Restrict egress and plugins for manifest-generation components.
- Define fail-open or fail-closed behavior for policy and secret-service outages.
- Separate emergency authority from ordinary contribution and reconcile every emergency action back into declared state.

Reference Configuration or Command Pattern

```
# policy-oriented release evidence
release_evidence:
  source_revision: <sha>
  image_digest: sha256:<digest>
  sbom: <uri>
  provenance: <uri>
  signature_verified: true
  manifest_policy: pass
```

```
admission_policy: pass
approver: <identity>
exception: null
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Secret appears in Git history	Plain value was committed before encryption	Revoke/rotate first, inspect use, then sanitize history where justified	Rotation and exposure investigation
Admission outage blocks all deploys	No failure-mode or emergency policy	Use tiered policy availability and controlled break-glass	Outage exercise and decision log
Signed image still unsafe	Signature proves identity, not quality or authorization	Verify provenance, policy, vulnerability and promotion context	Complete trust decision
Audit cannot explain release	Evidence is spread or overwritten	Centralize immutable release record linked by digest/revision	Reproducible release timeline

Hands-on Lab 39: Build a Governed GitOps Trust Chain

Prerequisites

- A disposable Kubernetes cluster with kubectl access.
- A Git repository and a sample container image in a registry.
- Permission to create a dedicated namespace and inspect controller logs.

Implementation

1. Create a threat model for repository, promotion, controller, registry, policy, secret manager, and cluster.
2. Replace one plaintext secret with an external reference and workload identity.
3. Add manifest validation, server dry-run, and admission policy with tests.
4. Verify an image digest and record signature or attestation evidence before reconciliation.
5. Create an expiring exception and prove it cannot outlive its scope.
6. Simulate policy-service and secret-service outages with documented behavior.
7. Build a dashboard and release-evidence record for one deployment.

Validation Evidence

- The declared revision and image digest are visible in Git and runtime evidence.
- Healthy-state and failure-state results are captured.
- Rollback or recovery meets the stated objective.

Cleanup

- Delete temporary namespaces, releases, repositories, credentials, and test records.
- Retain only sanitized diagrams, logs, decisions, and validation evidence.

Interview Questions and Model Answers

1. Why are encrypted secrets in Git not automatically safe?

Security depends on key custody, recipient scope, rotation, repository access, history, and where decryption occurs.

2. What is policy as code?

Versioned, reviewable, testable policy logic integrated into delivery and enforcement systems.

3. Does a signature prove an image is secure?

No. It provides identity/integrity evidence; authorization still depends on provenance, policy, findings, and context.

4. What should an exception contain?

Requirement, scope, owner, approver, evidence, compensating controls, expiry, monitoring, and closure criteria.

Chapter Summary

- GitOps security spans source through runtime.
- Secret, identity, policy, artifact, and controller boundaries must align.
- Observability must explain decisions and runtime outcomes.
- Exceptions and emergency changes are governed production objects.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 40: Enterprise Kubernetes Delivery Platform Capstone

Integrating packaging, GitOps, progressive delivery, security, observability and recovery into a portfolio-grade platform.

Why this chapter matters: The final Part D project proves that individual tools form one supportable delivery product with measurable value, bounded risk, and reproducible evidence.

Learning Objectives

- Integrate Part D capabilities into one enterprise architecture.
- Operate multiple teams, environments, and clusters through golden paths.
- Prove promotion, policy, progressive delivery, drift, rollback, and disaster recovery.
- Measure platform reliability and developer experience.
- Present and defend architecture trade-offs.

Core Concepts

Capstone scenario

A regulated digital-services company runs public APIs and background services across development, staging, production, and recovery clusters. Teams need self-service delivery while the platform group maintains identity, policy, artifact, telemetry, and recovery guardrails.

Required architecture

Create source and environment repositories, packaging standards, immutable registry promotion, Argo CD or Flux reconciliation, progressive delivery, secret references, policy, workload identity, dashboards, alerts, backup, and evidence storage.

Acceptance evidence

Provide requirements, diagrams, ADRs, threat model, RBAC matrix, repository protections, rendered manifests, policy tests, release record, rollout analysis, drift game day, rollback, controller rebuild, SLO report, cost estimate, and user feedback.

Operating model

Define platform consumers, service catalog, support tiers, on-call, ownership, SLO/error budget, upgrade policy, deprecation, exception process, and roadmap. A technically impressive platform without adoption and support is incomplete.

Architecture and Operating Decisions

- Keep the architecture deployable after every phase.
- Choose one primary GitOps controller and document why the alternative was not selected.
- Use at least two workload classes and one stateful dependency with explicit migration behavior.
- Automate evidence collection and require independent post-deployment verification.
- Run a final technical defense covering failure, security, cost, and organizational trade-offs.

Reference Configuration or Command Pattern

```
capstone/
  architecture/  # diagrams, ADRs, threat model
  platform/     # controller bootstrap and policies
  packages/     # Helm/Kustomize application contracts
  environments/ # promoted immutable desired state
  tests/        # render, policy, rollout and recovery
  observability/ # dashboards, alerts and SLOs
  runbooks/     # deploy, abort, rollback, restore, rebuild
  evidence/     # sanitized release and game-day records
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Golden path works only for demo	Operational ownership, quotas or failure paths were omitted	Run realistic team onboarding and incident exercises	User test and SLO evidence
Recovery depends on original cluster	Bootstrap inputs or credentials are not portable	Create clean-room rebuild from protected sources	New-cluster recovery result
Progressive delivery slows every release	Risk tiers and automatic low-risk paths are absent	Define release classes and proportional controls	Lead-time and failure metrics
Platform adoption is low	Interface, documentation or migration support misses team needs	Interview users, fix top friction and publish roadmap	Adoption funnel and satisfaction

Hands-on Lab 40: Deliver and Defend the Kubernetes Delivery Platform

Prerequisites

- A disposable multi-namespace or multi-cluster Kubernetes environment.
- Git hosting, container registry, CI system, secret manager or safe equivalent, and observability stack.
- A review panel or second engineer for independent validation.

Implementation

1. Define business outcomes, workload classes, environments, trust boundaries, SLOs, RTO/RPO, and cost constraints.
2. Build packaging and environment repositories with protected promotion of immutable images.
3. Deploy Argo CD or Flux with tenant boundaries, policy, secret delivery, workload identity, and observability.
4. Implement a rolling release and an automated canary with business and technical analysis.
5. Run drift, invalid configuration, unavailable dependency, policy outage, compromised credential, and controller-loss scenarios.
6. Restore service, roll back or fix forward, and rebuild the control plane in a clean environment.
7. Present architecture, evidence, residual risk, platform metrics, and roadmap to a review panel.

Validation Evidence

- A fresh reviewer can reproduce bootstrap and one application deployment.
- Every production claim links to a revision, digest, policy, health, or recovery record.
- Rollback, progressive abort, drift correction, and controller rebuild meet stated objectives.
- No secrets, private identifiers, or sensitive evidence are present in the portfolio.

Cleanup

- Destroy sandbox clusters, registries, identities, repositories, DNS, load balancers, storage, and test secrets.
- Verify cost-bearing resources and privileged credentials are removed.
- Archive only sanitized portfolio artifacts and lessons learned.

Interview Questions and Model Answers

1. What is the most important capstone outcome?

Reproducible evidence that delivery, security, operation, recovery, ownership, and user value work together.

2. How should Argo CD versus Flux be chosen?

Compare operating model, ecosystem, UX, extensibility, tenancy, scale, skills, and recovery needs against the scenario.

3. What makes a golden path successful?

It is safer and easier than bypassing it, handles common workload needs, is observable, documented, supported, and continuously improved.

4. What demonstrates senior delivery capability?

Explicit trade-offs, bounded risk, measured outcomes, truthful residual risk, and proven recovery under realistic failure.

Chapter Summary

- The platform integrates technical and organizational controls.
- Evidence must cover delivery, policy, health, recovery, cost, and adoption.
- Failure scenarios expose hidden dependencies.
- A final defense demonstrates engineering judgment, not tool memorization.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Part D Command and Investigation Reference

Purpose	Reference pattern
Inspect workload	kubectl get deploy,rs,pod,svc,ingress -n <ns> -o wide
Render Kustomize	kubectl kustomize overlays/<environment>
Render Helm	helm template <release> <chart> -f values-<env>.yaml
Rollout status	kubectl rollout status deployment/<name> -n <ns> --timeout=5m
Revision history	kubectl rollout history deployment/<name> -n <ns>
Argo CD sync	argocd app sync <app> && argocd app wait <app> --health
Flux reconcile	flux reconcile kustomization <name> --with-source
Flux status	flux get all -A
Argo Rollouts	kubectl argo rollouts get rollout <name> --watch
Events	kubectl get events -n <ns> --sort-by=.lastTimestamp
Server-side dry run	kubectl apply --server-side --dry-run=server -f <manifest>
Diff desired state	kubectl diff -f <manifest>

Production Readiness Graduation Checklist

- ☐ Desired state, artifact digest, policy bundle, environment, and promotion history are traceable.
- ☐ Controllers use least privilege and separate tenant or environment blast radii.
- ☐ Rendered manifests are validated before merge and again before admission.
- ☐ Drift, failed reconciliation, stale sources, unhealthy rollouts, and policy failures alert an owner.
- ☐ Rollback and fix-forward paths are tested with data and dependency implications documented.
- ☐ Secrets are delivered without plaintext storage in Git and rotation is rehearsed.
- ☐ Progressive delivery uses measurable analysis, abort thresholds, and business validation.
- ☐ Controller backup, restore, bootstrap, and clean-room rebuild evidence exists.
- ☐ Platform SLOs, support boundaries, quotas, capacity, and upgrade policy are published.
- ☐ Runbooks and dashboards can be used by an independent on-call engineer.

Ten Additional Kubernetes Delivery Portfolio Projects

Project	Primary evidence	Difficulty
1. Multi-environment application repository	base/overlay design, rendered diff and promotion history	Medium
2. Helm chart product	versioned chart, tests, schema and release notes	Medium
3. Argo CD application platform	AppProjects, ApplicationSets, SSO and recovery evidence	Advanced
4. Flux fleet bootstrap	GitRepository/Kustomization hierarchy and	Advanced

Project	Primary evidence	Difficulty
	bootstrap rebuild	
5. Progressive rollout	canary analysis, abort and rollback evidence	Advanced
6. Secret delivery pattern	rotation, denial and outage behavior	Advanced
7. Policy-controlled delivery	admission tests, exceptions and audit evidence	Advanced
8. Drift game day	unauthorized change detection and reconciliation report	Medium
9. Multi-cluster promotion	environment gates, cluster inventory and failover	Advanced
10. GitOps observability dashboard	controller SLOs, alerts and operational runbook	Medium

90-Day Kubernetes Delivery Practice Plan

Weeks	Focus	Required output
1-2	Kubernetes release primitives and packaging	deployment labs, Helm chart and Kustomize overlays
3-4	GitOps repository and promotion design	environment repository, ADRs and drift tests
5-6	Argo CD and Flux operations	two controller implementations and recovery evidence
7-8	Progressive delivery	canary, blue-green, analysis and abort report
9-10	Security, secrets and governance	policy bundle, secret rotation and tenant isolation
11	Troubleshooting and mock incidents	incident timelines and runbook improvements
12-13	Final capstone	portfolio-grade Kubernetes delivery platform and defense

Glossary

Term	Definition
ApplicationSet	Argo CD controller that generates Applications from declarative generators.
Canary	Progressive exposure of a new version to a bounded share of traffic or users.
Desired state	Versioned declaration of the intended runtime configuration.
Drift	Difference between declared desired state and observed cluster state.

Term	Definition
GitOps	Operating model in which versioned declarations and automated reconciliation drive change.
Health assessment	Controller evaluation of whether resources are operationally healthy.
Kustomize	Template-free Kubernetes configuration customization using bases and overlays.
Progressive delivery	Controlled release using staged exposure, analysis, and automated promotion or abort.
Reconciliation	Repeated controller action that moves observed state toward desired state.
Sync wave	Ordering mechanism for groups of Argo CD resources.
Tenant boundary	Isolation of permissions, repositories, namespaces, clusters, or controller scope.
Workload identity	Short-lived identity used by a workload without embedded static credentials.

Official Documentation Index

Kubernetes Documentation: <https://kubernetes.io/docs/>

Kubernetes Deployments: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

Kustomize: <https://kubectldocs.kubernetes.io/references/kustomize/>

Helm Documentation: <https://helm.sh/docs/>

Argo CD Documentation: <https://argo-cd.readthedocs.io/>

Argo Rollouts Documentation: <https://argo-rollouts.readthedocs.io/>

Flux Documentation: <https://fluxcd.io/flux/>

Gateway API: <https://gateway-api.sigs.k8s.io/>

OpenGitOps Principles: <https://opengitops.dev/>

Kubernetes SIG Security: <https://kubernetes.io/docs/contribute/participate/sig-list/#sig-security>

Verification note: Products, versions, certification blueprints, pricing, quotas, and feature status change. Check the current official documentation before production use or exam registration.

Volume 10 Completion

With Part D complete, Volume 10 now contains a coherent path from pipeline foundations through Jenkins and SaaS CI/CD platforms into Kubernetes GitOps delivery, software-supply-chain security, enterprise operations, certifications, interviews, and the final delivery-factory capstone.

Final action: Complete the labs, retain sanitized evidence, score at least 80% on the knowledge checks, and rehearse the recovery procedures before marking this part complete.